

TP: BENCHMARK CPU VS GPU SUR DES OPERATIONS VECTORIELLES

MASTER D'EXCELLENCE EN INTELLIGENCE ARTIFICIELLE

MODULE : SYSTEMES REPARTIS & PROGRAMMATION DISTRIBUEE

Réalisé par :

Mohamed Akrame Kraichy / groupe 2

Hajar En-najdy / groupe 2

INTRODUCTION ET OBJECTIFS

Ce TP met en œuvre un benchmark comparatif entre le processeur (CPU) et le processeur graphique (GPU) pour trois opérations fondamentales de l'algèbre linéaire : la somme de vecteurs, le produit scalaire et la norme euclidienne. L'objectif central n'est pas de simplement constater que le GPU est plus rapide — c'est de comprendre **pourquoi** et **à partir de quelle taille** l'accélération devient significative.

OBJECTIFS PÉDAGOGIQUES :

- Configurer et vérifier l'environnement GPU (CUDA) sur Google Colab.
- Comprendre la notion de **point de bascule** : seuil à partir duquel le GPU surpasse le CPU.
- Maîtriser les 3 réflexes essentiels du benchmark GPU : synchronisation, warm-up, répétitions.
- Mesurer et analyser le **speedup** (accélération) GPU/CPU pour différentes tailles de vecteurs (1K à 100M éléments).
- Produire et interpréter des graphiques log-log pour visualiser les performances.

CONTEXTE THÉORIQUE

Un GPU contient des milliers de petits cœurs de calcul (CUDA cores) conçus pour exécuter des milliers d'opérations en parallèle. Ce parallélisme massif n'est utile que si le volume de données est suffisamment grand pour occuper tous ces cœurs. Sur de petits vecteurs, le **coût fixe de lancement** d'un kernel CUDA (démarrage, scheduling) domine le temps de calcul réel. Le GPU semble alors plus lent que le CPU.

LES TROIS OPERATIONS ETUDIEES

1. SOMME D'UN VECTEUR

La somme consiste à additionner tous les éléments d'un vecteur :

- $\sum a_i$
- Données lues : 1 vecteur
- Caractéristique : opération principalement limitée par les accès mémoire (**memory-bound**), car le coût de calcul est faible par rapport au volume de données à lire.

2. PRODUIT SCALAIRE

Le produit scalaire entre deux vecteurs est défini par :

- $\sum a_i b_i$
- Données lues : 2 vecteurs
- Caractéristique : nécessite la lecture simultanée de deux vecteurs, ce qui double pratiquement le trafic mémoire par rapport à une simple somme.

3. NORME EUCLIDIENNE

La norme euclidienne mesure la longueur d'un vecteur :

$$\sqrt{\sum a_i^2}$$

Note : Un vecteur de 100M éléments float32 pèse $100M \times 4$ octets = 400 Mo. Le produit scalaire nécessite deux vecteurs 800 Mo. Vérifiez la mémoire GPU disponible avant d'aller au-delà.

ENVIRONNEMENT ET CONFIGURATION

Google Colab est la plateforme recommandée, car elle fournit gratuitement un GPU **NVIDIA T4** ou **A100** avec environ **15 Go de VRAM**, évitant ainsi tout problème d'installation locale de **CUDA**.

ÉTAPES DE CONFIGURATION

- Se connecter sur **colab.research.google.com** et créer un nouveau notebook.
- Activer le GPU : **Exécution** puis **Modifier le type d'exécution** puis **GPU (T4 ou A100)**.
- Vérifier la disponibilité GPU avec la **Cellule 1** du notebook.
- Si `torch.cuda.is_available()` retourne `False`, redémarrer la session et revérifier le runtime.

DESCRIPTION DES CELLULES DU NOTEBOOK

Le notebook est organisé en 10 cellules. Exécutez-les dans l'ordre et observez attentivement les affichages à chaque étape.

Cellule	Titre	Description
Cellule 1	Vérification de l'environnement	Vérifie que PyTorch détecte correctement le GPU à l'aide de <code>torch.cuda.is_available()</code> . Affiche le nom du GPU ainsi que la mémoire disponible. Si aucun GPU n'est détecté, l'ensemble du benchmark s'exécute uniquement sur le CPU.
Cellule 2	Tensors et transfert CPU ↔ GPU	Présente la création d'un tensor sur CPU puis son transfert vers le GPU avec <code>.to('cuda')</code> . Cette étape montre que les données doivent être placées sur le bon périphérique avant l'exécution des calculs.
Cellule 3	Fonction <code>benchmark()</code>	Définit la fonction de mesure des performances. Elle réalise un échauffement (<i>warm-up</i>) de 3 exécutions, effectue 10 mesures, synchronise le GPU avec <code>torch.cuda.synchronize()</code> et retourne la médiane des temps d'exécution en millisecondes.
Cellule 4	Opérations vectorielles	Implémente les trois opérations étudiées : somme (<code>torch.sum</code>), produit scalaire (<code>torch.dot</code>) et norme euclidienne (<code>torch.norm</code>).
Cellule 5	Tailles testées	Définit les tailles des vecteurs testés : 1K, 10K, 100K, 1M, 10M et 100M éléments. Calcule également la mémoire nécessaire pour chaque taille afin d'anticiper les limitations du GPU.
Cellule 6	Boucle de mesure complète	Partie principale du benchmark. Pour chaque taille de vecteur et chaque opération, mesure séparément les temps d'exécution sur CPU et GPU. Les données sont créées avant le chronométrage afin de mesurer uniquement le temps de calcul.

Cellule 7	Tableau de résultats et speedup	Construit un DataFrame Pandas contenant les résultats, génère un tableau croisé (<i>pivot table</i>) et calcule le facteur d'accélération (<i>speedup</i>) défini par : $\text{Speedup} = \text{Temps CPU} / \text{Temps GPU}$. Un speedup supérieur à 1 indique un avantage du GPU.
Cellule 8	Graphique Temps vs Taille	Produit trois graphiques en échelle logarithmique (un pour chaque opération) comparant les temps CPU et GPU en fonction de la taille des vecteurs. Le point de croisement des courbes représente le seuil à partir duquel le GPU devient plus performant.
Cellule 9	Graphique du Speedup	Affiche l'évolution du speedup selon la taille des données en échelle log-log. Une ligne horizontale à $y = 1$ permet de distinguer les zones où le CPU est plus rapide de celles où le GPU domine.
Cellule 10	Conclusions	Section d'analyse des résultats obtenus. Elle permet d'identifier le point de bascule CPU/GPU, le speedup maximal observé et les différences de comportement entre les opérations de somme, produit scalaire et norme euclidienne.

LES 3 RÉFLEXES ESSENTIELS DU BENCHMARK GPU

Ces trois précautions sont **indispensables** pour obtenir des mesures fiables. Sans elles, vos chiffres ne voudront rien dire.

REFLEXE 1 : SYNCHRONISER LE GPU

Le GPU est asynchrone : Python lance un calcul et n'attend pas qu'il soit terminé avant de continuer. Une mesure naïve mesurerait donc uniquement le temps de lancement du kernel, pas le temps de calcul réel.

```
torch.cuda.synchronize() # Force Python à attendre la fin du calcul GPU
```

REFLEXE 2 : FAIRE UN WARM-UP

La première exécution d'un kernel GPU inclut des initialisations JIT (compilation, allocation de contexte CUDA). Ces coûts faussent la mesure. On effectue quelques exécutions 'à blanc' avant de commencer à chronométrer.

```
for _ in range(3): # 3 exécutions de chauffe fonction()
```

REFLEXE 3 : REPETER ET PRENDRE LA MEDIANE

Les temps varient d'une exécution à l'autre (bruit système, cache état, scheduling). On répète la mesure plusieurs fois et on prend la médiane (plus robuste que la moyenne aux valeurs aberrantes).

```
temps.sort() return temps[len(temps) // 2] # médiane
```

TRAVAIL DEMANDE

EXÉCUTION ET COLLECTE DES RÉSULTATS

- Exécutez toutes les cellules dans l'ordre sur Google Colab avec GPU activé.
- Notez les valeurs du tableau pivot (Cellule 7) pour chaque opération et chaque taille.
- Identifiez visuellement le **point de bascule** sur les graphiques (Cellule 8).
- Relevez le **speedup maximum** observé et l'opération pour laquelle il est le plus élevé.
- Sauvegardez les graphiques produits (clic droit Enregistrer l'image).

CONSEILS ET QUESTIONS BONUS

CONSEILS PRATIQUES

- Si `torch.cuda.is_available()` renvoie `False`, allez dans **Exécution puis Modifier le type d'exécution** et sélectionnez GPU, puis redémarrez.
- Si vous obtenez une erreur de mémoire sur les vecteurs de 100M éléments, réduisez la liste `TAILLES` (retirez le dernier élément).
- Vérifiez bien que les données sont **sur le bon device** avant de lancer le benchmark : mesurer un calcul GPU avec des données CPU provoquera un transfert silencieux et faussera les résultats.
- Utilisez `del a_gpu`, `b_gpu` et `torch.cuda.empty_cache()` entre les tailles pour libérer la mémoire GPU et éviter les erreurs OOM (Out Of Memory).

QUESTIONS BONUS

- **Q** : Modifiez le benchmark pour utiliser **float64** au lieu de `float32`. Observez-vous une différence de speedup ? Pourquoi ?
- **Q** : Ajoutez une opération **multiplication matricielle** (`torch.mm`) sur des matrices carrées de taille $N \times N$. Comparez le speedup obtenu avec celui des opérations vectorielles.
- **Q** : Testez l'impact du nombre de répétitions sur la stabilité des mesures (essayez `n_repet=3, 10, 50`). À partir de combien de répétitions les résultats se stabilisent-ils ?

CONCLUSION

Ce TP vous a permis d'explorer concrètement le rapport CPU/GPU sur des opérations vectorielles fondamentales. Vous avez appris que le GPU n'est pas toujours plus rapide : sur les petits vecteurs, le coût de lancement du kernel masque le gain de parallélisme. Ce concept de **point de bascule** est central dans tout système de calcul distribué ou HPC.

La maîtrise du benchmarking rigoureux (synchronisation, warm-up, médiane) est une compétence transférable à tous les environnements de calcul haute performance, qu'il s'agisse de clusters MPI, de systèmes distribués ou d'architectures de deep learning.

